The Journal of Open Source Software

pytorch-ood: A Unified PyTorch Library for Out-of-Distribution Detection

Konstantin Kirchheim [1¶], **Theo Langer** [1], **and Frank Ortmeier** [1]

1 Otto-von-Guericke University Magdeburg, Germany ¶ Corresponding author

DOI: 10.21105/joss.10806

Software

- Review
- Repository
- Archive

Editor: Neea Rusch

Reviewers:

- @neerazz
- @eramakanth
- @shehan807

Submitted: 15 April 2026
Published: 28 October 2026

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Machine learning models, particularly deep neural networks, can produce unreliable predictions when they encounter data that differs from what they were trained on. Detecting such cases – commonly referred to as out-of-distribution (OOD) detection – is particularly important in safety-critical applications (Yang et al., 2024). Despite rapid progress in the field, practical experimentation remains fragmented. Implementations are often tied to individual papers, interfaces differ across methods, and evaluation setups can be difficult to reproduce.

`pytorch-ood` is a Python library that provides a unified framework for OOD detection in the PyTorch deep learning ecosystem (Paszke et al., 2019). It encompasses a broad range of detection methods, training objectives, datasets, pretrained models, and evaluation tools under a consistent interface. This allows users to apply different methods, compare them under shared conditions, and integrate them into existing workflows with minimal additional code. The library is designed to support flexible experimentation rather than a single fixed benchmark. It is accompanied by extensive documentation and unit tests, enabling reliable reuse and facilitating reproducible research.

Statement of Need

Research on OOD detection faces a recurring methodological problem: many published methods are conceptually comparable but difficult to compare in practice. Small implementation differences in preprocessing, score orientation, or metric computation can materially affect conclusions. At the same time, reproducing baselines often requires re-implementing substantial amounts of auxiliary code for dataset handling, evaluation, and model integration. This duplication slows progress and makes empirical results harder to audit.

`pytorch-ood` was developed to reduce this friction. The package provides a unified API for detectors and related components while covering a broad range of methods and benchmark datasets. This enables the reuse of trained models across detectors, evaluation under a shared interface, and the integration of new methods without rebuilding surrounding infrastructure.

The library is intended for researchers developing and evaluating OOD detection methods in PyTorch. It reduces engineering overhead compared to bespoke per-paper implementations while remaining more flexible than fixed benchmark pipelines. In this sense, `pytorch-ood` provides a unified and reproducible interface for OOD detection experiments, bridging the gap between ad-hoc research code and standardized benchmarking frameworks.

An earlier version of the library was introduced in Kirchheim et al. (2022). The present work reflects a substantially extended and matured system.

State of the Field

Several tools for OOD detection exist, many of which were released after `pytorch-ood`. Frameworks such as OpenOOD (Yang et al., 2022) emphasize standardized benchmark pipelines, enabling consistent large-scale evaluation across datasets and methods. Other tools, such as FrOoDo (Stieber et al., 2022), focus on specific application domains, for example medical imaging.

`pytorch-ood` targets a complementary use case. Rather than prescribing a fixed evaluation pipeline or domain, it provides a flexible, PyTorch-native layer for composing detectors, models, datasets, and benchmarks under a unified interface. This design is motivated by the observation that exploratory research often requires fine-grained control over experimental details, access to intermediate neural representations, and varying training objectives, which can be difficult to achieve within fixed benchmark frameworks.

As a result, `pytorch-ood` is particularly well-suited for baseline construction, ablation studies, and the rapid integration of new methods. It is therefore best understood as complementary to benchmark-centric frameworks, prioritizing modularity, reuse, and controlled experimentation.

Software Design

The design of `pytorch-ood` is guided by the goal of enabling *comparable and reusable OOD detection experiments* while maintaining flexibility for research. A central design choice is interface unification.

Detectors

All OOD detection methods implement a shared `Detector` abstraction with a `predict()` method that maps inputs to outlier scores and, where required, an optional `fit()` method for calibration or training. This defines a minimal contract: given an input tensor x, a detector produces a tensor of outlier scores $s(x)$, where larger values indicate stronger evidence that the corresponding inputs or spatial locations are out-of-distribution. For classification, this typically yields one score per input sample, while segmentation methods may instead produce spatially resolved score maps. As a result, different detectors can be applied interchangeably and compared directly, without requiring knowledge of their internal implementation or score computation. The trade-off is a slight restriction on method-specific interfaces in exchange for consistent and comparable usage across detectors.

Representation-Specific Interfaces

OOD detectors can be differentiated based on the kind of input that they work on. Usually, they operate on neural representations of the input in some layer of a deep neural network. To further support reuse, the library distinguishes detectors by the type of representation they consume. In addition to the base interface, specialized classes such as `LogitsDetector` and `FeaturesDetector` operate on logits or intermediate feature representations, respectively. This design reflects a second trade-off: for different detectors that operate on the same type of representation, the required representations only need to be computed once rather than once per detector:

```
# reuse logits across detectors
logits = model(x)
scores1 = detector1.predict_logits(logits)
scores2 = detector2.predict_logits(logits)
```

While this separation reduces redundant computation and simplifies controlled comparisons, as multiple methods can be evaluated under identical conditions, it comes at the cost of slightly

increased API complexity.

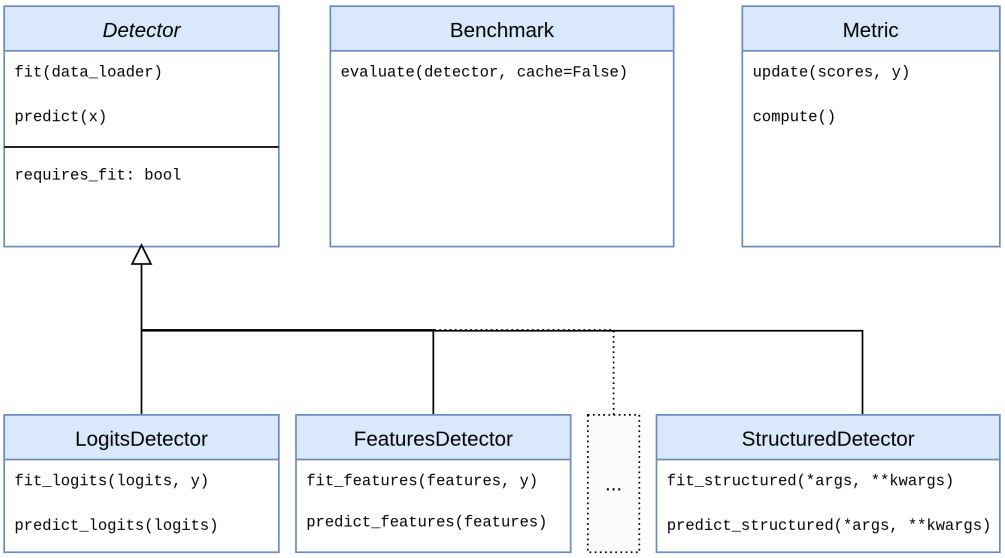


Figure 1: Overview of Relevant Abstractions

Evaluation Semantics

A third design principle is the standardization of evaluation semantics. In practice, OOD detection experiments often differ in label encoding or metric assumptions, which can lead to subtle and hard-to-detect errors in evaluation.

To mitigate this, `pytorch-ood` enforces a canonical evaluation protocol across all components. Datasets follow a unified label convention with OOD samples assigned $y < 0$, and metrics operate under consistent assumptions (such as the score orientation) across detectors and benchmarks. While this slightly reduces flexibility, it eliminates ambiguity in metric computation and ensures that results are directly comparable across methods.

Benchmarks

Finally, the library provides a benchmark abstraction that exposes a common `evaluate()` interface. This enables the reproduction of established evaluation protocols while remaining compatible with custom training setups. The design balances two competing goals: supporting standardized experiments and preserving the ability to compose new ones. In practice, benchmark objects allow users to evaluate multiple detectors on shared datasets with minimal boilerplate while retaining full control over other experimental details.

For example, a minimal evaluation workflow to replicate the OpenOOD v1.5 CIFAR-10 benchmark (Zhang et al., 2023) with a pretrained WideResNet-40-2, the maximum softmax probability baseline detector (Hendrycks & Gimpel, 2017), and a more recent detector (Liu et al., 2023) can be written directly as:

```
import torch

from pytorch_ood.detector import MaxSoftmax, GEN
from pytorch_ood.model import load_model, load_transform
from pytorch_ood.benchmark import CIFAR10_OpenOOD

device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
model_id = "wrn-40-2/cifar10/crossentropy"
model = load_model(model_id).to(device)
preprocess = load_transform(model_id)

msp = MaxSoftmax(model)
gen = GEN(model, gamma=0.1)

benchmark = CIFAR10_OpenOOD(root="data", transform=preprocess)

metrics = benchmark.evaluate(
  [gen, msp],
  loader_kwargs={"batch_size": 64},
  device=device,
  cache=True
)

print(metrics)
```

The model weights and the OpenOOD CIFAR-10 benchmark datasets are downloaded automatically. Since both detectors used in this example implement the `LogitsDetector` interface, logits will be extracted from the deep neural network only once and then passed through both detectors.

Research Impact Statement

The `pytorch-ood` library has seen sustained adoption within the research community. As of 7 September 2026, Google Scholar reported 72 citations to the associated publication (Kirchheim et al., 2022), GitHub listed 346 stars and 11 contributors, and PePy recorded 76,545 total downloads from PyPI.

AI Usage Disclosure

Generative AI tools were used during the development of `pytorch-ood` and the preparation of this manuscript. In particular, we used systems from OpenAI (including ChatGPT and Codex) and Anthropic (Claude) for parts of the codebase, documentation, and manuscript text. The exact historical model versions used for this assistance cannot be recovered. These tools provided assistance with code generation and refactoring, drafting and improving documentation, generating testing scaffolding, and editorial support. All AI-assisted contributions were carefully checked and integrated by the authors. All architectural and design decisions were made by the authors, who take full responsibility for the software and this manuscript.

Acknowledgements

The authors acknowledge institutional support from Otto-von-Guericke University Magdeburg, Germany. No dedicated financial support was received specifically for the development of `pytorch-ood` or for this JOSS submission.

References

Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.1610.02136

Kirchheim, K., Filax, M., & Ortmeier, F. (2022). PyTorch-OOD: A library for out-of-distribution detection based on PyTorch. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 4351–4360. https://doi.org/10.1109/cvprw56347.2022.00481

Liu, X., Lochman, Y., & Zach, C. (2023). GEN: Pushing the limits of softmax-based out-of-distribution detection. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 23946–23955. https://doi.org/10.1109/cvpr52729.2023.02293

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*. https://doi.org/10.48550/arXiv.1912.01703

Stieber, J., Fuchs, M., & Mukhopadhyay, A. (2022). FrOoDo: Framework for out-of-distribution detection. *arXiv Preprint arXiv:2208.00963*. https://doi.org/10.48550/arXiv.2208.00963

Yang, J., Wang, P., Zou, D., Zhou, Z., Ding, K., Peng, W., Wang, H., Chen, G., Li, B., Sun, Y., & others. (2022). OpenOOD: Benchmarking generalized out-of-distribution detection. *Advances in Neural Information Processing Systems*, *35*, 32598–32611. https://doi.org/10.52202/068431-2362

Yang, J., Zhou, K., Li, Y., & Liu, Z. (2024). Generalized out-of-distribution detection: A survey. *International Journal of Computer Vision*, *132*(12). https://doi.org/10.1007/s11263-024-02117-4

Zhang, J., Yang, J., Wang, P., Wang, H., Lin, Y., Zhang, H., Sun, Y., Du, X., Li, Y., Liu, Z., Chen, Y., & Li, H. (2023). OpenOOD v1.5: Enhanced benchmark for out-of-distribution detection. *NeurIPS 2023 Workshops: DistShift*. https://doi.org/10.48550/arXiv.2306.09301